Deployment

> Part of the MCP Server curriculum. This document covers transports in production, per-boundary auth decisions (with a worked four-boundary example using edge service tokens), connecting local, desktop-app and programmatic clients, and independent versioning.

1. Local-only vs. remote

The right transport is a direct consequence of where the server's clients actually run:

- **stdio** — the host launches the server itself as a subprocess. Appropriate whenever every consumer runs on the same machine as the server (a desktop LLM application, local development, or manual verification while iterating on tool descriptions). This has **no network exposure at all** — a real security property, not just a convenience, since there is no listening port to secure in the first place.
- **Streamable HTTP** — needed the moment any consumer runs somewhere else: a separately deployed agent process, a teammate's machine, a production client. At that point the server is a normal long-lived network service, and normal web-service deployment concerns apply directly — process supervision, a reverse proxy, TLS termination. Most of the API backend curriculum's deployment document transfers essentially unchanged.

2. Auth is a per-boundary decision, not one setting

An MCP server commonly sits between **two distinct trust boundaries** that don't need — and shouldn't automatically get — the same treatment:

Boundary	What it is	Typical treatment
Outbound — the server's own calls to whatever it wraps	The server acting as a client of an upstream API, database, or file store	Often fine unauthenticated if that upstream is already network-isolated (e.g. reachable only on localhost); revisit once the upstream itself requires credentials
Inbound — MCP clients reaching the server itself	Whoever/whatever connects to this server to call its tools	The boundary that actually changes once the server is reachable from anywhere other than the same machine — needs a real access-control decision

Don't assume solving one boundary automatically covers the other, and resist the instinct to add authentication code "to be safe" on a boundary that's already closed by network topology (e.g., a call that only ever travels over localhost) — a token with nothing on the other end to validate it is dead code, not defense in depth.

For the inbound boundary specifically, an **edge/gateway-level access control layer** (a managed access proxy sitting in front of the server, gating which external clients may reach it at all) is often a better fit than an application-layer token the server's own code has to validate — it keeps the server's code simpler, and keeps the access policy centrally managed outside the application rather than duplicated inside it.

2.1 A worked example: four boundaries, four decisions

Concretely, a public deployment of an MCP server in front of a data API has more than two legs, and each one gets its own answer:

Boundary	Parties	Can hold a secret?	Decision
Browser ↔ dashboard / API	Anonymous visitors ↔ public static app and API	No — anything a browser sends is visible in DevTools	Stay unauthenticated; the data is public. Protect with edge rate limiting, not a token the browser would have to ship
Server ↔ the API it wraps	Same host	Yes	No change — network isolation (bound to loopback) already does the job a token would do; a token with nothing to validate it is dead code
External MCP clients ↔ the server	A desktop LLM app, named testers, now internet-reachable	Yes — the client list is small and known	**Edge access policy with named service tokens** (below)
The project's own co-located agent ↔ the server	Same host	Yes	Treat as the loopback case, **not** the external-client case — it never leaves the machine

A single shared gate on the API would authenticate the second leg correctly and break the first (the browser cannot hold the secret), which is why the legs must not share one mechanism.

Edge access with service tokens. Put a managed access proxy in front of the MCP path with a *service-auth* policy (machine clients, not a login) and issue **one named, individually revocable token per client** (the desktop app, the tester, a future external agent). Per-client tokens make revocation and attribution a dashboard action rather than a redeploy, and the check happens at the edge before a request reaches your machine at all. Verify it from *outside* the host: no credentials and a wrong token must both be refused (403). Remember the accept path needs a real token to verify, which is easy to leave unverified — do it once the first real client is configured.

Three transport-level gotchas that authentication does not cover:

1. **Route order is load-bearing.** If the dashboard is served by an unanchored prefix rule (`/app`) and the MCP route is `/app/mcp`, the MCP route must sit *above* it in the tunnel's rule list, or every MCP request is swallowed by the dashboard's catch-all and answered with its `index.html` (HTTP 200) — no error, just a confusing client failure. Anchor patterns (`^/app/mcp`) and test with a real request.
2. **DNS-rebinding protection rejects a deployment left at its defaults.** The MCP SDK's transport security defaults to an empty allowed-hosts list; behind a tunnel every request carries the public hostname and is refused (421 Misdirected Request). Pass the allowed hosts explicitly — *including the loopback host and port a co-located consumer connects on*, which is easy to forget because it is "authenticated" by topology yet still a different `Host` header. Apply the setting only when deployed (not `None` vs. an *empty* settings object — the latter still rejects every local request).
3. **A path prefix must be normalised once.** If the public path prefix comes from the same environment variable the API and frontend read, run it through the same normalisation (strip trailing slash) before appending `/mcp`, or you get `/app//mcp`.

3. Connecting clients

- **Local/manual verification clients** — a desktop LLM application or an IDE-integrated coding assistant, configured directly against a stdio (or local HTTP) server. This is the fastest way to iterate on tool names, descriptions, and argument schemas by hand, and is worth doing **before** any downstream agent code exists at all — it's the direct way to verify tool-calling quality in isolation, rather than debugging it through an extra layer of agent orchestration.
- **A desktop LLM app against a header-authenticated remote server.** Many desktop apps' graphical "add a custom connector" flow supports only OAuth 2.1 — not arbitrary static headers. A server gated by edge service tokens (rather than implementing OAuth itself) will fail that flow in a confusing way: the app opens a browser to an `/authorize` URL that does not exist, and it downloads as an empty file. The working path is to skip the graphical flow and add a config-file entry that runs a small local **stdio↔HTTP bridge** which injects the two access headers itself, taking the secrets from an `env` block rather than inlining them. Use the token issued specifically for that client so revocation stays meaningful. Expect such a bridge to be the less stable link in the chain: if the connector shows "failed" after having worked, relaunching the desktop app is the first fix, and a server-side keepalive on the event stream is the likely root-cause fix.
- **A programmatic client** — an agent process connecting over Streamable HTTP, typically through a client-adapter library that manages the session lifecycle (the initialize handshake, keeping the connection alive, converting tool schemas into whatever shape the calling LLM API expects) rather than hand-rolling the protocol. See the conversational-agent curriculum for what this looks like from the client side.

4. Independent versioning and deployability

Treat an MCP server as its **own deployable unit** — its own dependency manifest, separate from whatever it wraps — the same separate-deployability principle that applies to a decoupled frontend. This lets the server's tool-schema releases move on a different cadence than the underlying data source or API, and keeps its own dependencies (an MCP SDK, a fuzzy-matching library, and similar) from leaking into unrelated parts of a larger codebase that has no reason to depend on them.

5. Co-locating with its consumer

When a server's only real consumer is one specific agent deployed alongside it, running both on the same host and treating that connection as an already-network-isolated boundary (no app-layer auth needed on that leg, per §2) is a reasonable simplification, not a shortcut to feel guilty about — it's the correct read of the actual trust boundary in that topology. Revisit it only if a second, less-trusted consumer shows up later and needs to reach the same server from somewhere that boundary no longer covers.

6. Deployment checklist

Concern	Decide
Transport	stdio if every client is local; Streamable HTTP the moment any client isn't

Concern	Decide
Outbound auth	Only if the upstream itself requires it — not "to be safe"
Inbound auth	Edge/gateway access control for external reachability, rather than a hand-rolled app-layer token, where the two are equivalent choices
Local verification	Connect a desktop/IDE LLM client directly before building anything downstream
Versioning	Own manifest, deployed independently of whatever the server wraps
Co-location	Fine to treat as network-isolated when the only consumer is co-located; revisit if that changes
Per-boundary table	Write down every leg (browser, server→API, external clients, co-located agent) and decide each separately (§2.1)
Edge route order & allowed hosts	MCP route above any catch-all; allowed hosts include the loopback a co-located consumer uses (§2.1)
Desktop clients	Header-authenticated remote servers need a local bridge, not the OAuth-only connector UI (§3)

See also

- MCP Server curriculum index
- Core Concepts, §5 for the transport tradeoffs this document builds on
- API Backend Deployment for the process supervision/reverse proxy/CI details that transfer directly to a Streamable HTTP deployment
- Conversational Agents, Deployment for the matching decisions on the agent side of this connection